Database Design – Exercise 1 Answers

Prepared from the lecture slides “Part 1: Introduction to Database Systems”

Question 1 — Short answers

1. Two disadvantages of a database system compared with a file system

- **Higher cost and complexity:** a DBMS is heavy, expensive software that needs stronger hardware and a database administrator (DBA). Example: for a small app's contact list, setting up a DBMS is not worthwhile.
- **Performance overhead and concentrated risk:** concurrency control, transactions, security and logging slow things down, and a DBMS failure can stop every application that depends on it. Example: processing one large text file once with a simple program is usually faster than doing the same job through a DBMS.

2. A real system where the file-based approach is justified

Web-server log files: data is only appended to the end of the file, has a simple sequential structure, needs no complex queries or relationships, and few users modify it concurrently. Other examples: an application's configuration files, and storing photos and videos.

3. Real examples of data by structure

- **Structured:** a students table with columns student number, name, major (fixed, predefined format).
- **Semi-structured:** JSON or XML files (self-describing), e.g. a web API response or an e-mail with headers and body.
- **Unstructured:** a recorded class video, an image, an audio file, or the free text of an article.

4. Controlled vs. uncontrolled redundancy

- **Uncontrolled (undesirable):** duplication caused by the non-database approach, managed by different systems unaware of each other. Example: a student's data is stored both in the Education Office file and the Student Affairs file, and a change in one is not applied to the other.
- **Controlled (acceptable):** deliberate duplication that the DBMS knows about and keeps consistent. Two kinds: natural (e.g. the same major shared by several students) and technical (e.g. indexing or backup copies).

5. Indexing and technical redundancy

An index (e.g. a B-tree or B+ tree) keeps the value of the search-key field, such as the student number, together with a pointer to the record in a separate structure. The key value is therefore stored both in the data and in the index, which is a controlled, technical redundancy. It is still useful: instead of scanning all records linearly, a search takes a few disk accesses through a multi-level tree. The cost is extra space and slower insert, delete and update, because the index must be updated too.

6. Semantic vs. logical vs. physical design

- **Semantic (conceptual modeling):** independent of any DBMS or implementation; identifies the entities, attributes, relationships and constraints of the real world (e.g. an ER model).

- **Logical:** maps the semantic model to the data model of the target DBMS (e.g. tables, keys and normalization in the relational model), without storage details.
- **Physical:** how data is stored on disk: file structures, indexes, blocking and performance tuning.

7. Are UI and application design part of a database project?

Yes. In the database approach, the “database project” includes, besides semantic, logical and physical design, UI design, application design (with transaction definition) and development of programs that define, control and process the data. These are the upper layer of the project, built on top of the database design rather than being the core of the database itself.

8. Data model vs. its implementation, and why it matters

- **Data model:** an abstract set of concepts (structure, relationships, constraints and operations) for describing data; e.g. the relational model, proposed as a theory in 1970.
- **Implementation:** a specific DBMS (e.g. MySQL or Oracle) that realizes the model with particular storage structures and algorithms (relational implementations appeared in the 1980s).
- **Importance:** the separation provides program–data independence (abstraction). The implementation can be optimized or replaced without changing the model or the applications, designers can focus on concepts, and different products can be compared against one shared model.

9. Which item is NOT a component of a data model?

Data Sequence. The other three — Data Semantic, Data Relationship and Data Constraint — are components of a data model.

10. Which component prevents two people from registering for a course with one free seat?

Transaction management and concurrency control in the DBMS. With locking, two simultaneous registration transactions run in isolation (the Isolation property): the first fills the seat, the second sees the course full and is rejected. The integrity constraint “enrolled ≤ capacity” also plays a part.

11. Withdrawal from A1, then an error before the transfer to A2

The property involved is **Atomicity**: a transaction must either complete entirely or leave no effect at all. When the error occurs, the withdrawal from A1 is rolled back so that the total A1 + A2 stays unchanged. Consistency is related too, but the direct cause and main property is Atomicity.

12. Effect of the snapshot frequency

The more frequently snapshots are taken:

- **Normal-operation performance:** decreases, since every snapshot costs I/O, processing and storage.
- **Recovery speed after a crash:** increases, since the last snapshot is closer to the failure and fewer changes must be replayed.
- **Information lost:** decreases, since the gap between the last snapshot and the failure is shorter.

A low snapshot rate has the opposite effect: better normal performance, but slower recovery and more lost data.

13. Why is disk primary and magnetic tape secondary storage?

A disk offers direct (random) access and returns a record in a few milliseconds, which suits online queries and transactions. Tape offers only sequential access, so reaching one record requires a long scan. In exchange, tape has a much lower cost per bit (CPB), high capacity and durability, and is portable, so it suits backups and archives (secondary storage).

14. Effect of rebuilding a replacement disk: Disk Mirroring vs. Striping+Parity

- **Disk Mirroring (e.g. RAID 1 and RAID 10):** rebuilding is just copying blocks from the healthy mirror disk to the new disk. Only one disk is read, no computation is needed, the load on the other disks is small, and the rebuild is fast, so the running system loses little performance.
- **Striping+Parity (e.g. RAID 5 and RAID 6):** to rebuild each block, all remaining disks must be read and the data recomputed from parity (e.g. XOR). I/O and CPU load are high, the process is long, and system performance drops significantly. During this time every read that would hit the failed disk also needs computation. The risk of a second disk failing in this window is also higher, which in RAID 5 means data loss.

15. Which applications suit a centralized architecture?

Applications whose data and users are mostly in one place, with moderate data volume and load, and where unified control and security matter; e.g. a company's accounting and inventory system, a faculty's education system, or a library's loan system. Management and consistency are easy, but there is a single point of failure and a performance bottleneck.

16. Main reason for the 3-tier architecture in web applications

Separation of the presentation tier (browser and UI), the application-logic tier (web server and application) and the data tier (DBMS). As a result each tier can be changed and scaled independently, maintenance is simpler, users never access the database directly (better security), database connections are pooled and shared, and nothing but a browser needs to be installed on the client.

17. A logically integrated but physically distributed database

Users and applications see one schema and one unified view (a global schema), but the data is spread over several servers or sites (via fragmentation or replication), and the distributed DBMS hides the real location of the data through “distribution transparency”. Example: a university database with three sites (Tehran, Mashhad, Shiraz) where the user sees a single “Student” table, while the records are stored on three servers.

Question 2 — Redundancy and inconsistency in two student files

When the Education Office and the Student Affairs Office each keep a separate file with the fields student number, name, major, address and phone, every student is stored twice and the two copies drift apart with no automatic coordination.

Scenario	What happens	Type of problem
Duplicate storage	All fields of every student are kept in both files; space is doubled, and every change must be entered twice by two people.	Undesirable redundancy

Change of address or phone	The student reports the new address only to Student Affairs; Education keeps the old one, so letters and transcripts go to the wrong address.	Update inconsistency
Change of major	The major is corrected in Education but stays old in Student Affairs; welfare services and statistics are computed from the wrong major.	Update inconsistency
Uncoordinated insert or delete	A new student is recorded in only one file, or a withdrawn student is removed from only one file; the student “exists” in one office and “does not exist” in the other.	Insert/delete inconsistency
Different formats	One name is spelled two ways (“Alireza” vs. “Ali Reza”) or a phone number is written in two formats; matching records and joint reports across the two offices become impossible or error-prone.	Data-format inconsistency

How does a database architecture reduce these problems?

- **Data integration:** each student is stored once in a shared table (e.g. Student(StdNo, Name, Major, Address, Phone) with primary key StdNo). Data specific to each office lives in separate tables that reference StdNo. Uncontrolled redundancy disappears.
- **Single-point update:** an address or major change is made once and both offices immediately see the new value.
- **Integrity constraints:** the primary key prevents duplicates, and domain and format constraints (e.g. phone format) block inconsistent values.
- **Transactions and concurrency control:** simultaneous updates from both offices run atomically and in isolation, so no intermediate inconsistency appears.
- **Different views:** each office sees only its own columns and rows (with access control) without keeping a separate copy.
- **Controlled redundancy:** any necessary duplication (indexes, backup copies) is coordinated and kept consistent by the DBMS itself.

Question 3 — Comparing RAID levels with 8 equal disks

Assumption: each disk has capacity S, so raw capacity is 8S. RAID 1 with 8 disks would be an 8-way mirror; if 4 mirrored pairs are meant, that is RAID 10.

Level	Usable capacity	Fault tolerance	Read performance	Write performance	Rebuild complexity
RAID 0	8S (100%)	None; one failed disk destroys the whole array	Excellent (parallel over 8 disks)	Excellent	No rebuild; restore from backup only
RAID 1	≈ S (12.5%)	Up to 7 disks	High (read from any copy)	Same as one disk (every write goes to all copies)	Simple and fast (copy from a healthy copy)
RAID 5	7S (87.5%)	1 disk	Good	Weaker (a small write ≈ 4 I/O operations because of parity)	Complex and long (read all 7 remaining disks)
RAID 6	6S (75%)	2 disks simultaneously	Good	Weaker than RAID 5 (two parities)	More complex and longer (two parities to compute)
RAID 10	4S (50%)	At least 1 disk; up to 4 if only one disk of each pair fails (depends on the failure pattern)	Excellent	Good (two writes, no parity computation)	Simple and fast (copy from the mirror of the same pair)

Choosing a level for three applications

Application	Recommended level	Reason
Data warehouse (read-heavy, large volume, batch loading)	RAID 6 (RAID 5 if cost matters more and regular backups exist)	Most usable capacity after RAID 0 and good reads; the write penalty does not matter for batch loads. With large disks, rebuilds are long, and two-disk tolerance covers the risk of a second failure.
Critical financial system	RAID 10 (RAID 6 if tolerance of any 2 failures must be guaranteed)	Priority is fault tolerance and fast, simple rebuilds, which shorten the window of risk, and writes carry no parity penalty; the cost is 50% of the capacity. RAID is not a substitute for backups, which are still needed.
High-traffic transactional system (OLTP)	RAID 10	Many small random reads and writes; RAID 5 and RAID 6 slow down under this pattern because of the parity write penalty, while RAID 10 gives high performance and low latency.

In practice both the financial system and OLTP end up on RAID 10, but for different reasons: fault tolerance and simple rebuilds for the financial system, write performance for OLTP.

Question 4 — Data distribution in a three-site database (Tehran, Mashhad, Shiraz)

General rule: data that is mostly used locally is fragmented; small, read-heavy, rarely changed data is replicated; sensitive data or data needing unified control is centralized.

Data	Decision	Availability and performance	Distribution transparency	Inconsistency risk
Student registration	Horizontal fragmentation by the student's home campus (plus a backup copy of each fragment at another site)	Most accesses are local and fast; if one site fails, the backup copy at another site stays available.	Needs fragmentation transparency: the user sees one “Registration” table and the DBMS finds the real location of each record.	Low; every record has one owning site. Only cross-site queries (e.g. an inter-campus course) need a distributed transaction.
Basic course information	Full replication at all three sites	Highest availability and read speed: every site reads it locally and one site failing causes no problem.	Replication transparency: the user does not know which copy is read.	Low to medium; the data is small and rarely changes, so updates are propagated from one master site to the others.
Financial activity reports	Centralized (e.g. in Tehran) with a backup copy at another site	Access from Mashhad and Shiraz is slower and depends on Tehran and the network; the backup reduces this risk.	Location transparency: users in other cities obtain reports through one interface without knowing where the data is stored.	Very low; there is one authoritative copy. Security control, auditing and report consolidation are also simple.
Course content (slides, videos)	Fragmentation by the courses offered at each site, with selective replication for shared or popular courses	Each site serves its own courses' content quickly and locally, and bulk data does not travel over the inter-site network; replicas of shared courses raise their availability.	Full distribution transparency: a student reaches the content through a single address.	Low; updates are usually made by the course instructor at one site, and only the selective copies must be synchronized.

Summary of considerations

- **Replication:** raises availability and read speed, but increases the risk of inconsistency and the cost of synchronization and space; suitable only for small, rarely changed data.
- **Centralization:** removes inconsistency and complexity, but lowers availability and performance for remote users; suitable for sensitive and aggregated data.
- **Fragmentation:** improves locality of access and scalability, but makes cross-site queries expensive and complex; suitable for large data with a local access pattern.